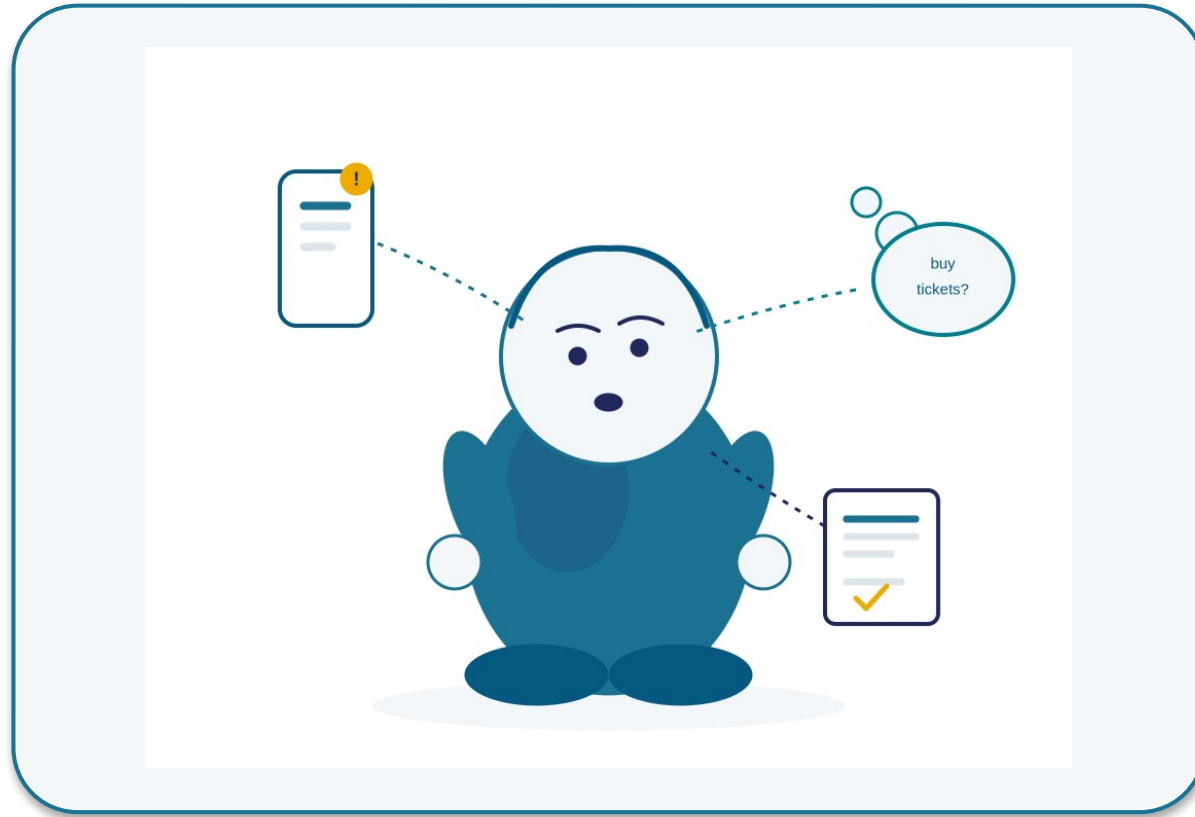


As you read this sentence — what are you NOT paying attention to?

Every second your brain chooses what matters and what is background.



The mechanism we take apart inside an AI model today has exactly the same name — "attention". It works in a similar spirit — but not the way yours does.

How modern large models work

4 inference stages: tokenization · embedding · attention · sampling



Lecture map — 6 sections

0

Introduction

*What a token is +
the central question*

1

Tokenization

*How the model sees
your text*

2

Embeddings

*The space
of meanings*

3

Attention

*What matters
now*

4

Sampling

*From distribution
to token*

5

Wrap-up

*Closing,
bridge to Lec 3*

Today we zoom into the "model" layer of Lecture 1's four layers

(from Lecture 1)

Application

Agent

Chat

MODEL — we zoom in today

What we know:

Model = stateless inference. Input is data, output is a prediction. No memory between calls.

What we learn today:

What's inside inference. 4 stages:
tokenization -> embedding -> attention -> sampling

The central question of the lecture

"What happens inside an LLM between my request and its answer - and which of these mechanisms change how I use it?"

3 answers — the wrap-up of the lecture

1

Why does a prompt with a role work better than an empty one?

→ *Section 3 — attention*

2

Why is AI bad at counting letters?

→ *Section 1 — tokenization*

3

Why does one request give different answers?

→ *Section 4 — sampling*

Section 1

Tokenization

How the model sees your text

0 Intro

1 Tokens

2 Embeddings

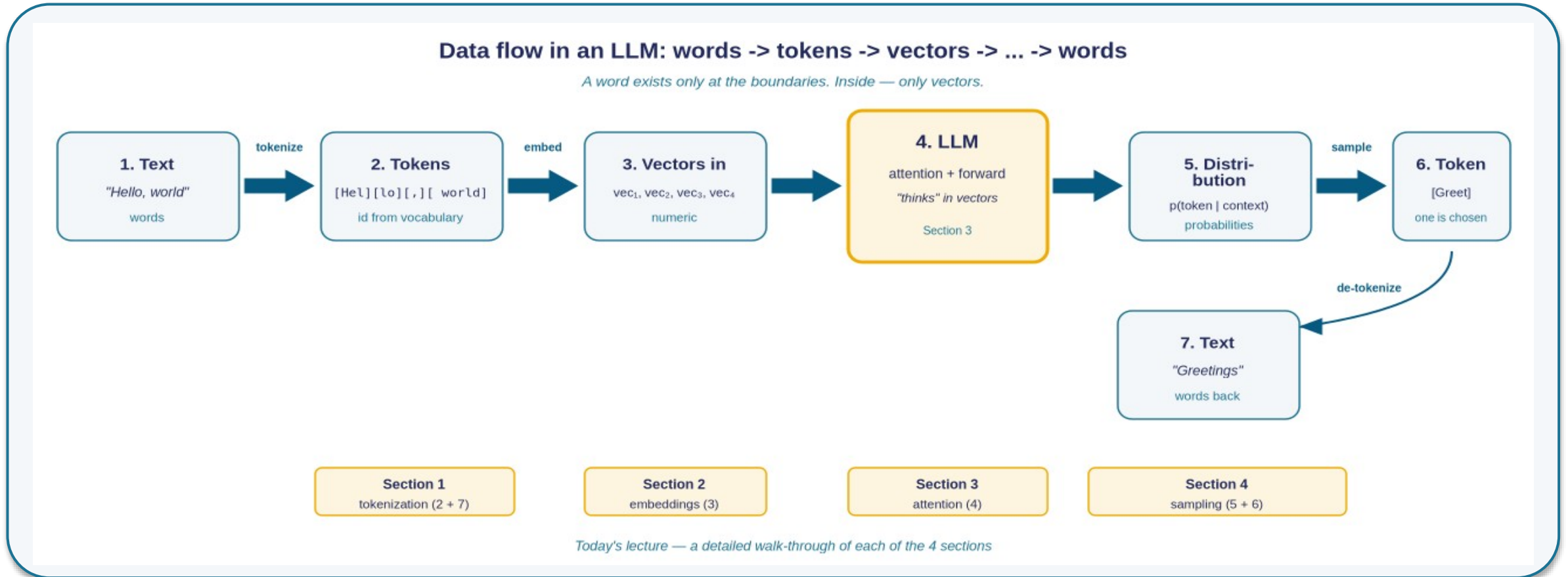
3 Attention

4 Sampling

5 Wrap-up

Data flow in an LLM — there and back

We'll cover each stage separately. Now — the whole map.



A word exists only at the boundaries. Inside the model — only vectors. Today we cover 4 stages.

A token is an id from the model's vocabulary. Not a letter and not a word.

A statistically frequent subsequence

Example 1.

cat

→

[cat]

1 token / 1 id

Example 2.

tokenization

→

[token]

[ization]

2 tokens

Example 3.

strawberry

→

[st]

[raw]

[berry]

3 tokens ·
o200k_base

On average: 1 token ~ 4 characters in EN ~ 2 characters in RU

For Russian, inference costs about twice as much — we'll return to this in 2 slides.

BPE — a compromise between an alphabet and a dictionary

The vocabulary is not made of all words (like lemmatization) nor of all letters (like character-level) — but of frequent subsequences.^[1]

The vocabulary is built once before training; at inference it is a lookup of ready-made rules.

Before (training corpus)

- low
- lower
- newest
- widest



After (BPE vocabulary)

- low
- er
- new
- est
- wid

The BPE vocabulary is built once before training. At inference it is a lookup of ready-made rules, not a runtime computation.

[1] [Sennrich et al. \(2016\)](#) — NMT of Rare Words / BPE

AI errs on "how many r-s in strawberry" — words are made of 3 tokens, not letters

strawberry in o200k_base (GPT-4o)

10 letters — what a human sees:

s t r a w b e r r y



3 tokens — what the model sees:

st raw berry

The model sees 3 units — not 10 letters

Counting characters

"How many r-s in strawberry?" — breaks systematically, non-obvious to the user.

Typos

methodology $\mapsto$ different tokens than methodology. A small typo $\rightarrow$ a big shift in the answer.

Case and spaces

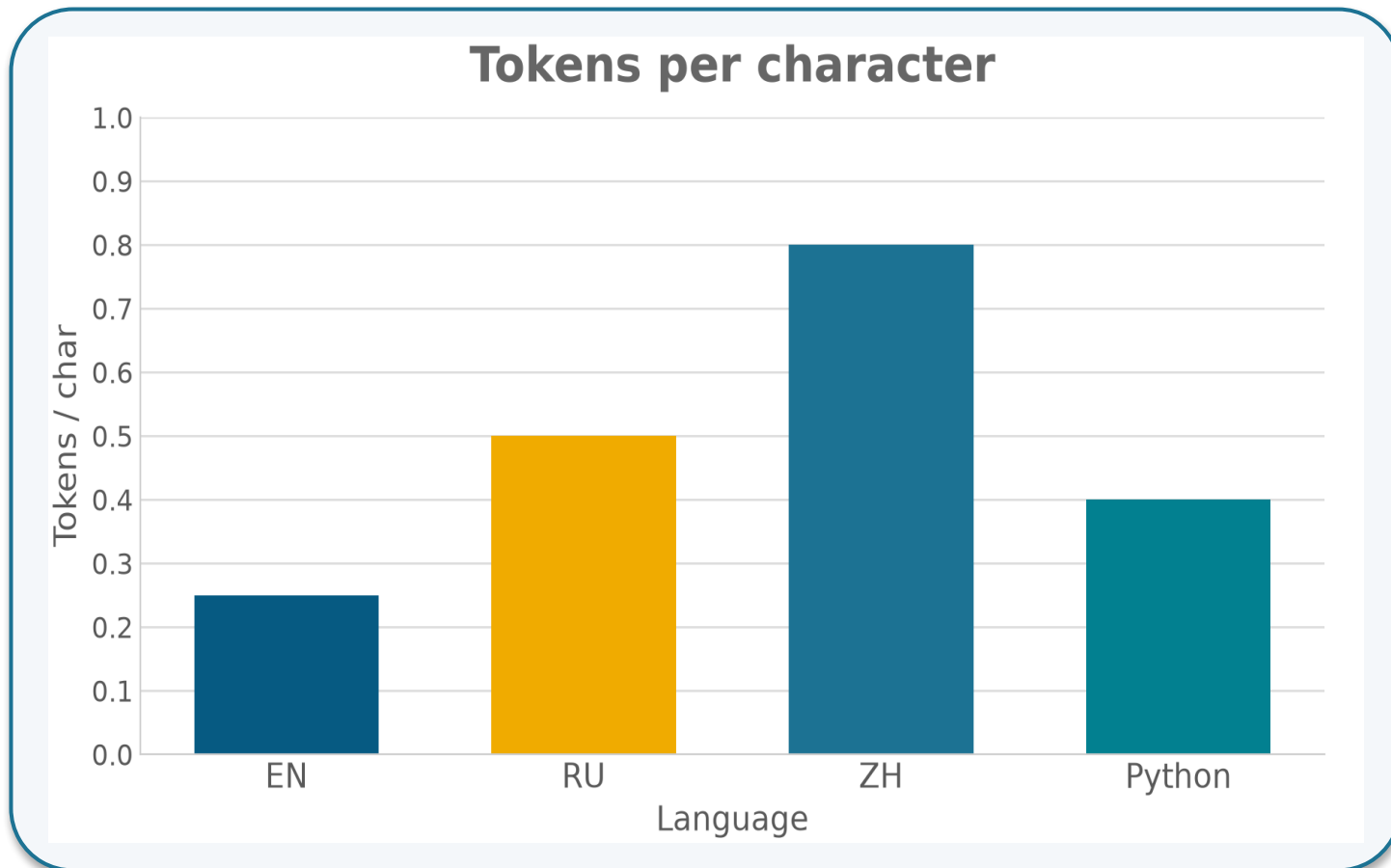
cat, `cat`, Cat, CAT — different tokens, different ids.

Letters: an external tool (Python, regex) or a character-by-character request. Top 2026 models call code themselves instead of a single pass.

Numbers: also split unpredictably (1234 $\rightarrow$ 12+34, but 55688 $\rightarrow$ 556+88) $\rightarrow$ GPT-4: 59% accuracy on 3-digit multiplication, 4% on 4-digit, 0% on 5-digit without a calculator.^[1]

The same text in Russian costs 2x more than in English

Tokens per character — a consequence of the language mix in the training corpus



Rule of thumb: tokens/char

English	0.25
Russian (gold)	0.50
Chinese	0.80
Python code	0.40

API cost for RU ~ 2x EN. For batch — translate to EN where acceptable.

Section 2

Embeddings

The space of meanings

0 Intro

1 Tokens

2 Embeddings

3 Attention

4 Sampling

5 Wrap-up

Each token maps to a vector — learned during training, fixed

Embedding: id -> learned vector

[cat]

id = 47284

lookup in
embedding table →

[0.21, -0.45, 0.88, ..., 0.13]

several hundred to several thousand dimensions
learned during training, fixed at inference

Geometric closeness = semantic closeness

Dimensions (rule of thumb)^[1]

text-embedding-3-small

1536 dim

text-embedding-3-large

3072 dim

Internal embedding of a flagship LLM

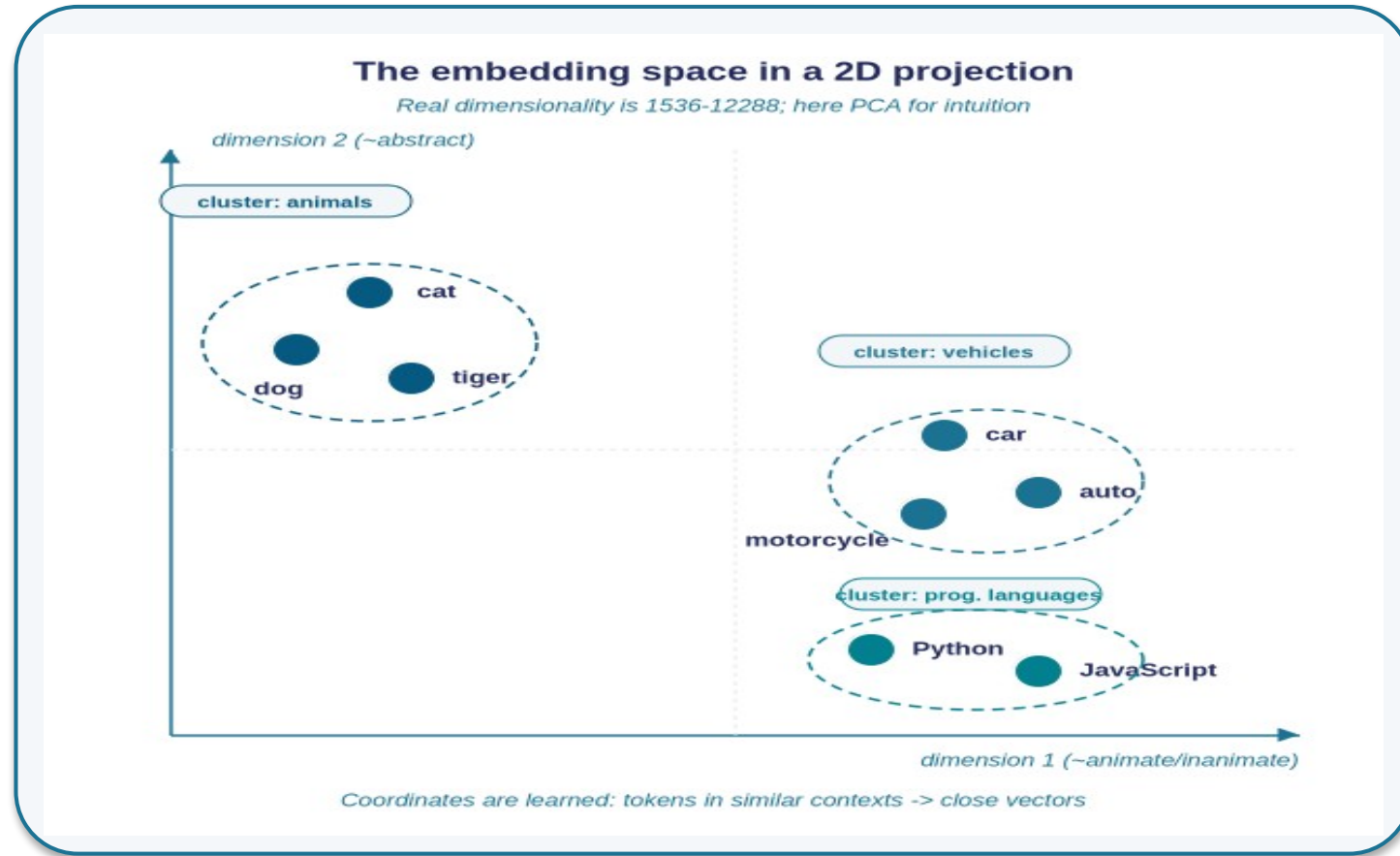
thousands dim

Geometric closeness of vectors = semantic closeness
of tokens.^[2]

"cat" is close to "dog" — learned from the corpus contexts.

The embedding space — where the vectors live

Before comparing vectors — let's look at what space they live in.



1 Dimensionality

OpenAI text-embedding-3-small: 1536 dim.
text-embedding-3-large: 3072 dim.

Flagship LLM: thousands.

2 Learning

Coordinates are not set by hand. The model learns:
tokens in similar contexts → close vectors.

3 Projection

To see the space — PCA or t-SNE
reduce the dimensionality to 2D/3D.

Some of the structure is lost.

Semantic similarity = geometric closeness of vectors. On the next slide — the measure.

Closeness in the embedding space = semantic similarity

In 2026 this works at the sentence level, not only for words

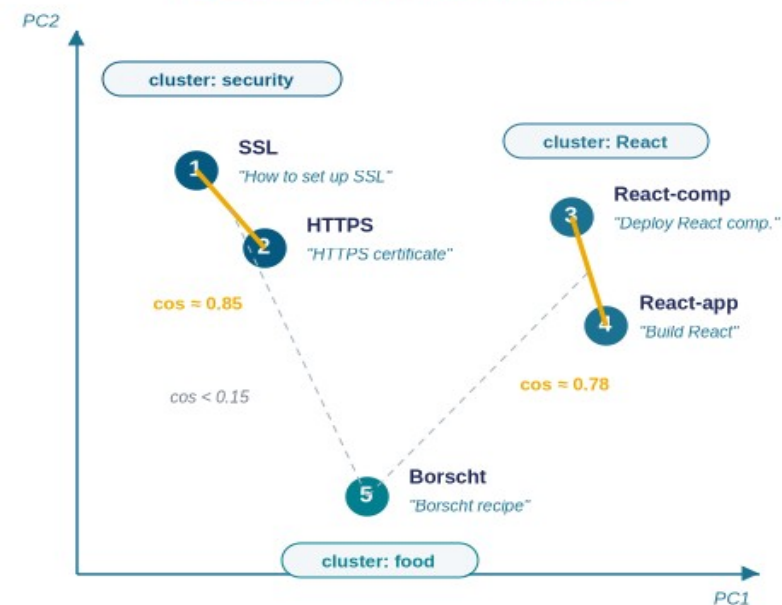
Cosine similarity — 5 sentences

Cosine = angle between vectors, [0, 1]; closer to 1 = more similar

	SSL	HTTPS	React comp.	React app	Borscht
SSL	1.00	0.85	0.18	0.20	0.08
HTTPS	0.85	1.00	0.22	0.19	0.07
React comp.	0.18	0.22	1.00	0.78	0.12
React app	0.20	0.19	0.78	1.00	0.10
Borscht	0.08	0.07	0.12	0.10	1.00

5 sentences in a 2D projection

PCA: 384-dim -> 2D, clusters visualize semantics



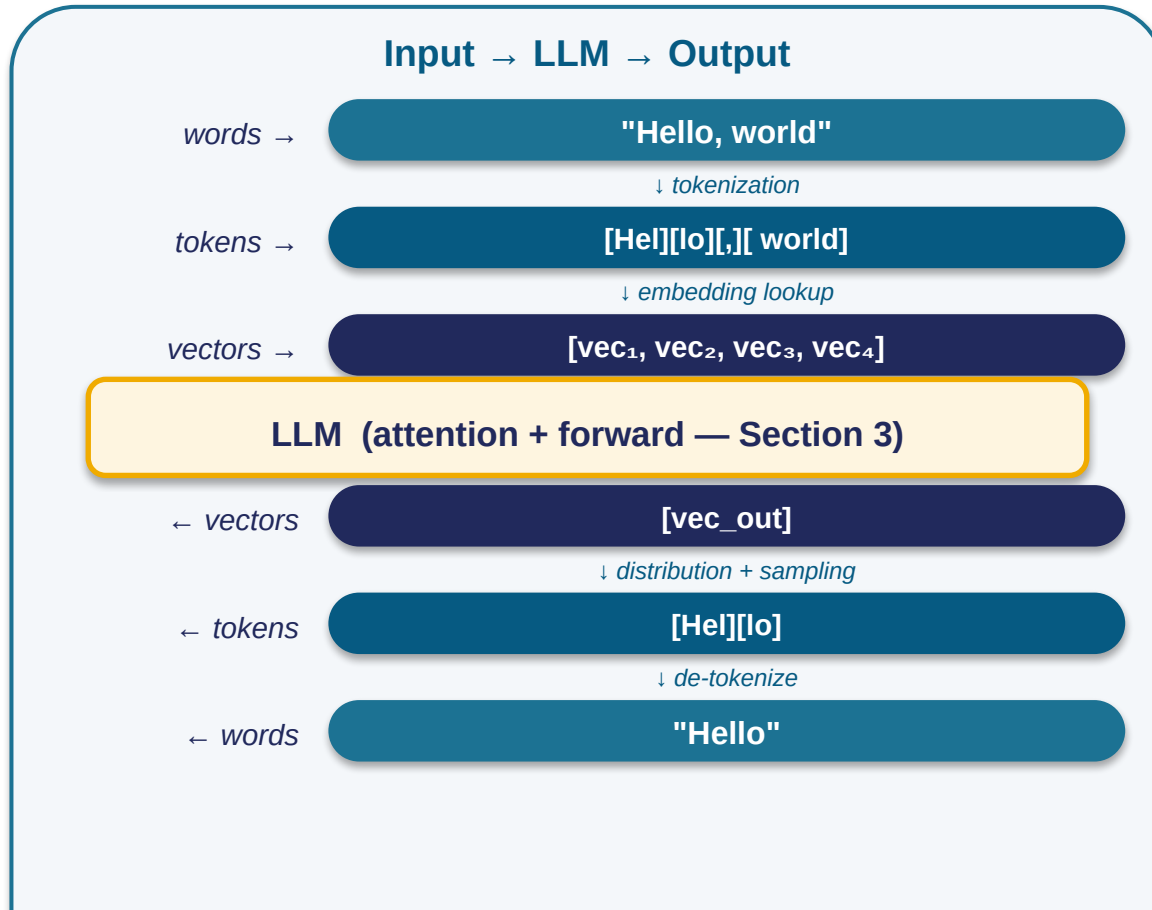
Close points = close meaning. Lines — pairs with cos > 0.7.

Cosine similarity — a measure of the angle between vectors; range [-1, 1], closer to 1 = more similar. Numbers are illustrative.^[1]

[1] [reproducible: all-MiniLM-L6-v2 / text-embedding-3-small](#)

Embeddings — the foundation of LLM understanding

Inside, the model works only with vectors; words appear only at the input and the output.



What gives it "understanding"



Paraphrases

"How to set up SSL" and "Installing HTTPS" → close vectors → the same answer.



Synonyms

"auto" and "car" → close vectors → the same reaction.



Cross-lang

"klubnika" (RU) and strawberry → close vectors → a correct answer in any language.

Sentence-level semantic similarity is the basis of an LLM "understanding" reformulations.^[1]

Section 3

Attention mechanism

How the model decides what matters now

0 Intro

1 Tokens

2 Embeddings

3 Attention

4 Sampling

5 Wrap-up

Attention is each token checking against every other token

Each token "looks at" all the others at once. At every step — $N \times N$ links.

Attention matrix 7x7 (single-head, simplified)
"The cat ate the mouse, because she was hungry"

	cat	ate	mouse	be- cause	she	was	hungry
cat	1.0	0.3	0.2	0.1	0.1	0.1	0.0
ate	0.4	1.0	0.5	0.1	0.1	0.1	0.1
mouse	0.2	0.4	1.0	0.1	0.1	0.0	0.1
because	0.1	0.2	0.2	1.0	0.3	0.2	0.2
she	0.1	0.1	0.7	0.2	1.0	0.3	0.4
was	0.1	0.2	0.2	0.3	0.4	1.0	0.5
hungry	0.1	0.2	0.3	0.3	0.4	0.5	1.0

"she" attends to "mouse"
weight = 0.7 (gold cell)

$N \times N$ links; for 100k
— ~10 billion numbers

1 Dimensionality

$N \times N$, where N is the context length.
Context 100k → a matrix of ~10 billion numbers.
→ the quadratic cost of attention.

2 At every step

The matrix is recomputed at every
new generated token (not once).

3 Multi-head

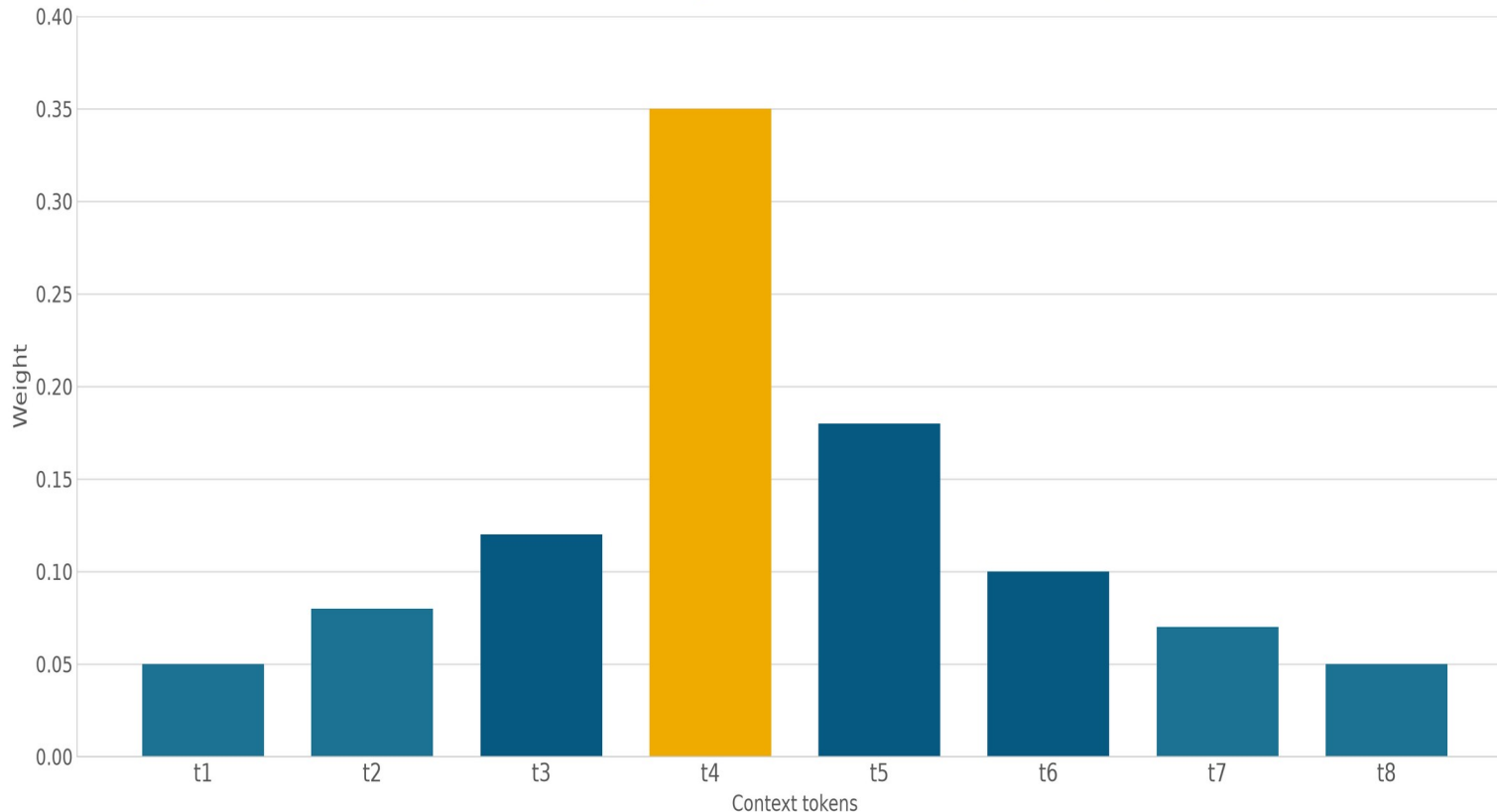
In one layer — dozens of such matrices
in parallel. Different "heads" look at
grammar, topic, distance.

Attention is a matrix operation, not a linear one. Each token is compared with all the others.

Attention outputs a weight distribution over all context tokens (sum = 1)

Which tokens matter now for predicting the next one^[1]

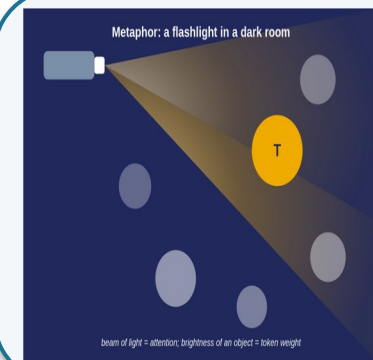
Attention weight distribution (sum = 1)



No formulas. Multi-head, Q/K/V — further reading.

3 properties

- 1 Input — all context tokens.
- 2 Output — a distribution, $\Sigma = 1$.
- 3 Recomputed at every step.



*Metaphor:
a flashlight in a
dark
room — the model
"lights up" some
tokens brighter
than others.*

Role tokens get elevated weight in the attention weights

Part A — a worked example; Part B — the role effect (1st of 3 "whys")

A. Worked example — where "she" looks

"The cat ate the mouse, because she was hungry"

she → mouse

thick · main weight

she → was

medium

she → hungry

thin

Simplification: a real attention map has hundreds of links. The model looks statistically, it does not do grammatical parsing.

Without a role

"Explain asynchrony"

→ *a generic answer*

(low weight of role tokens in the attention weights)

With a role

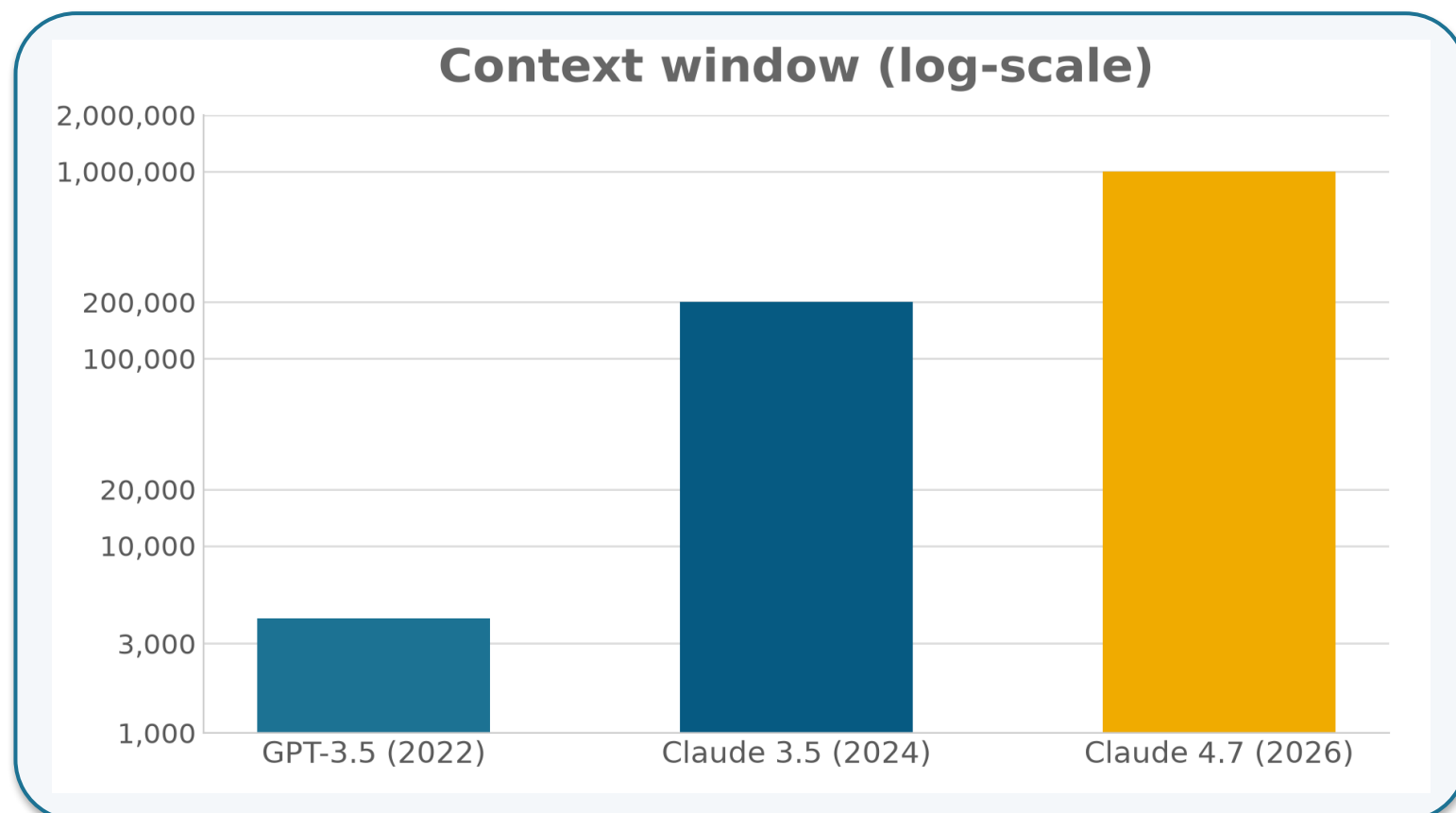
"You are **a Python expert**. Explain asynchrony **to a junior**."

→ **role tokens are highlighted**

(high weight in the attention weights)

The context window — a physical limit on how much the model sees at once

Context-window evolution + the quadratic cost of attention



Evolution and cost

2022 → 2026: ×250 growth

Pace: ×10 / 1-2 years

Cost N^2 : 1M ≈ 16× of 100k

Architecture: base attention

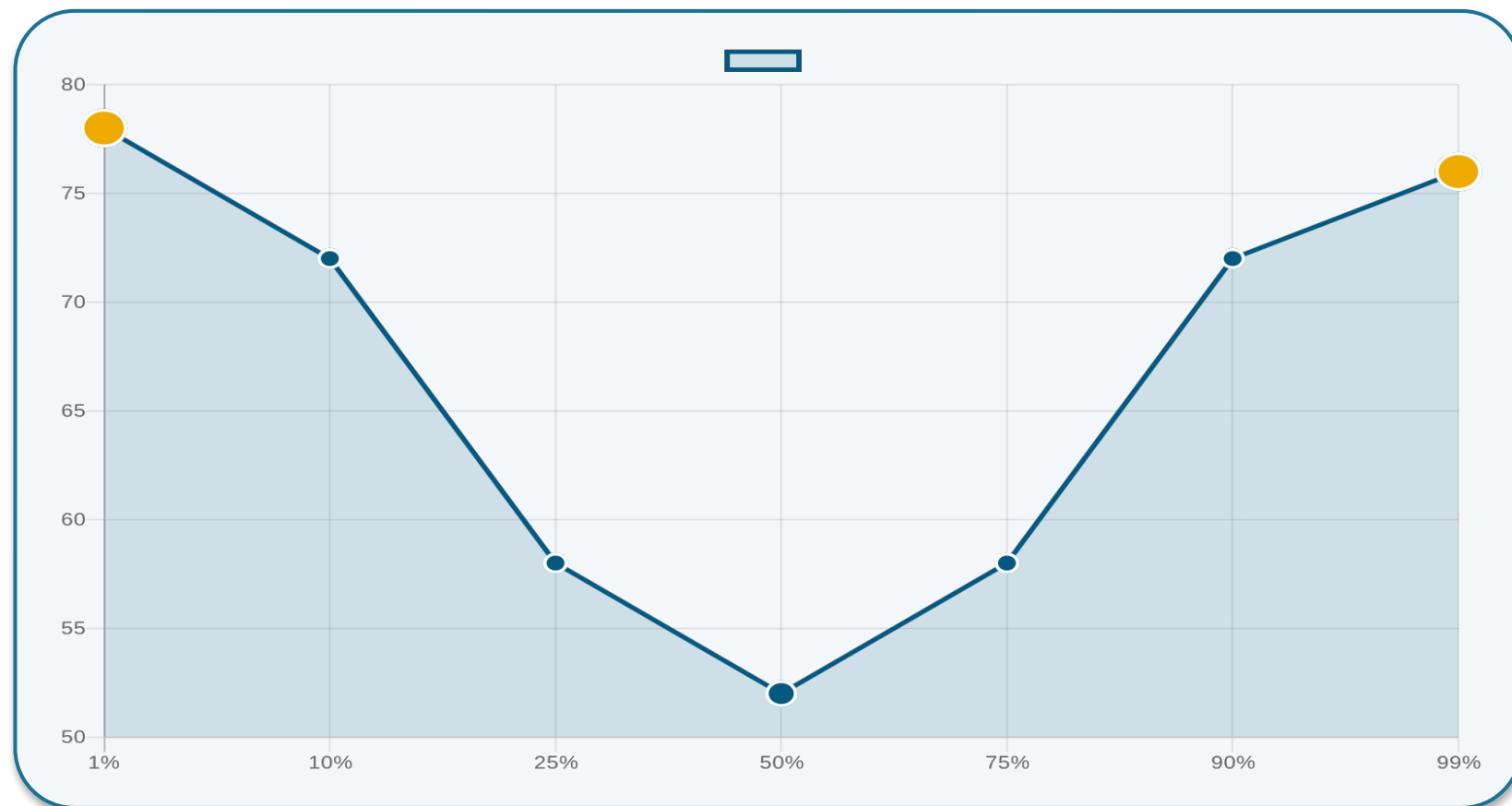
N^2

Attention cost grows quadratically with length. 1M ~ 16x more than 100k — production pricing with batching; pure N^2 -theory would give 100x.^[1]

[1] Anthropic — context window of the Claude models

A large context window != good use of the context

Lost-in-the-middle effect — the model forgets the middle^[1]



Experiment

A fact is placed at position X (start / middle / end) of a 100k context. The model answers about the fact.

Results

Start: ~75%

Middle: ~50%

End: ~75%

*Liu et al. 2023.
Lost in the Middle.*

Engineering takeaway: put what matters at the start or the end of the prompt, not in the middle.

[1] [Liu et al. \(2023\) — Lost in the Middle](#)

Section 4

Sampling

From the distribution to a token

0 Intro

1 Tokens

2 Embeddings

3 Attention

4 Sampling

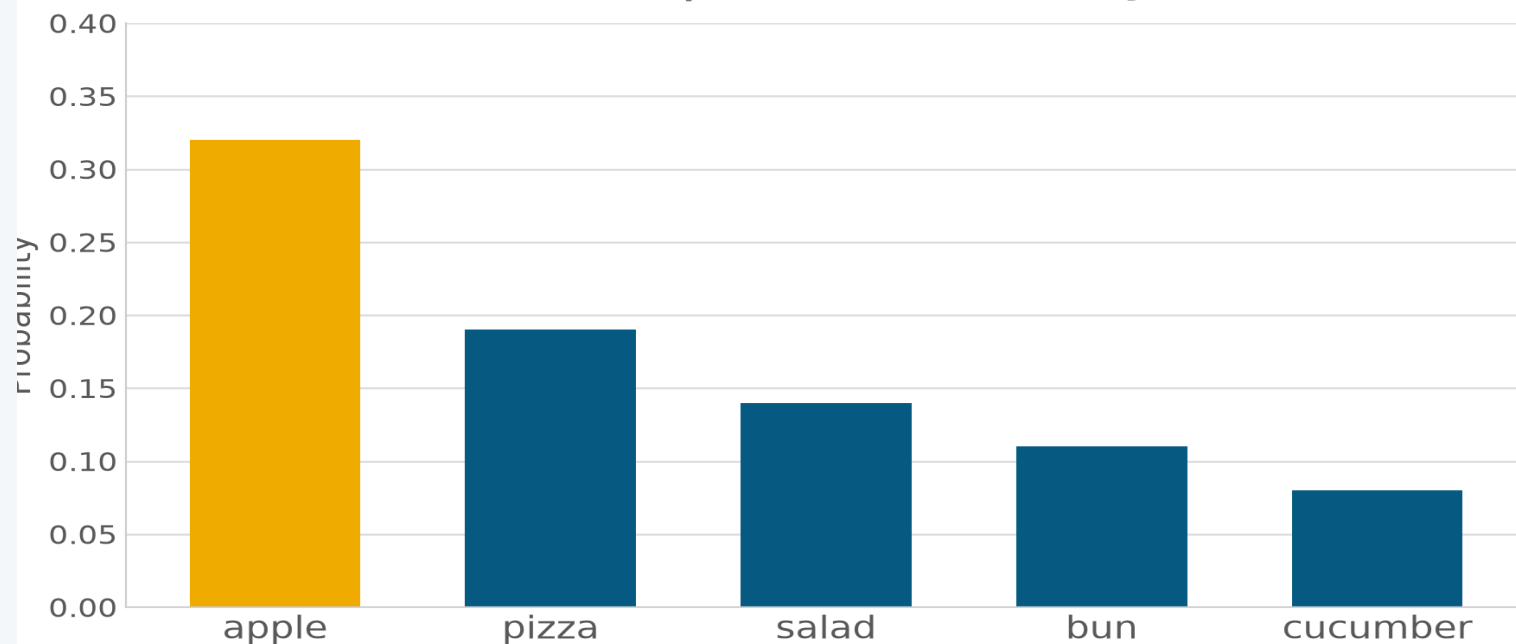
5 Wrap-up

At each step the model outputs a probability distribution over all tokens — and picks one

Context

"Today I ate ..."

P(next token) | context: "Today I ate..."



Top-5 candidates

apple	0.32
pizza	0.19
salad	0.14
a bun	0.11
cucumber	0.08

... the other ~200k tokens:
each < 0.05

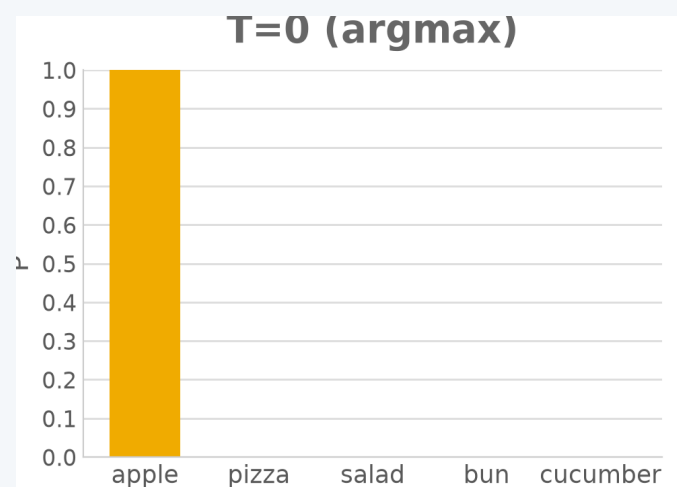
$\Sigma = 1$

Sampling = the rule by which one token is picked from the distribution. Next — temperature.

Temperature: how sharp the choice will be

$T = 0$ (argmax) · $T = 1.0$ (standard) · $T = 2.0$ (chaos)

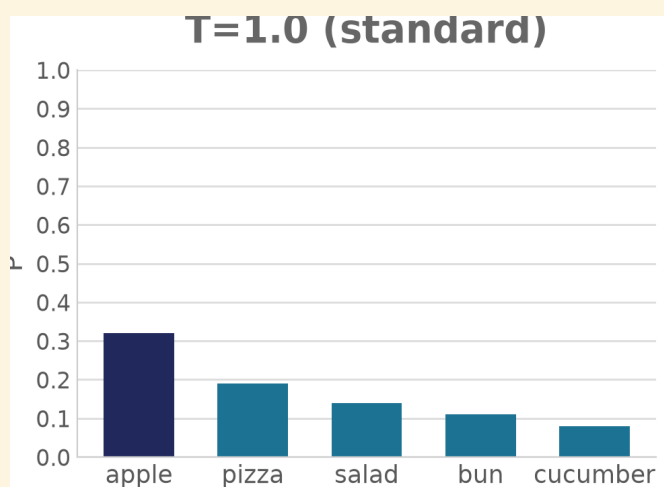
$T = 0$ · argmax



*Deterministic
choice — apple.*

10 runs → identical.

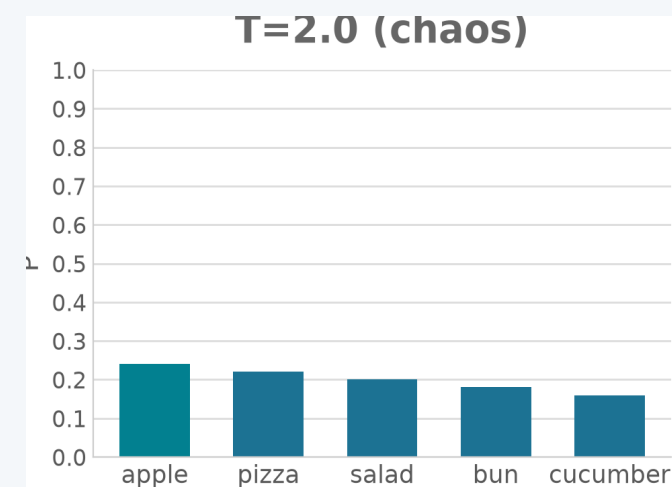
$T = 1.0$ · standard



*Sampling
proportional to P .
Natural variability.*

($T = 0.7$ — the standard choice for chat)

$T = 2.0$ · chaos



*The distribution is flattened;
unexpected options are
often chosen.*

Alternative knobs: top-p (nucleus) — cuts rare tokens by Σ ; top-k — by the number of candidates. T is enough to start. ^[1]

[1] [Holtzman et al. \(2019\) — nucleus sampling \(top-p\)](#)

4 API knobs for the task: temperature, top_p, max_tokens, system prompt

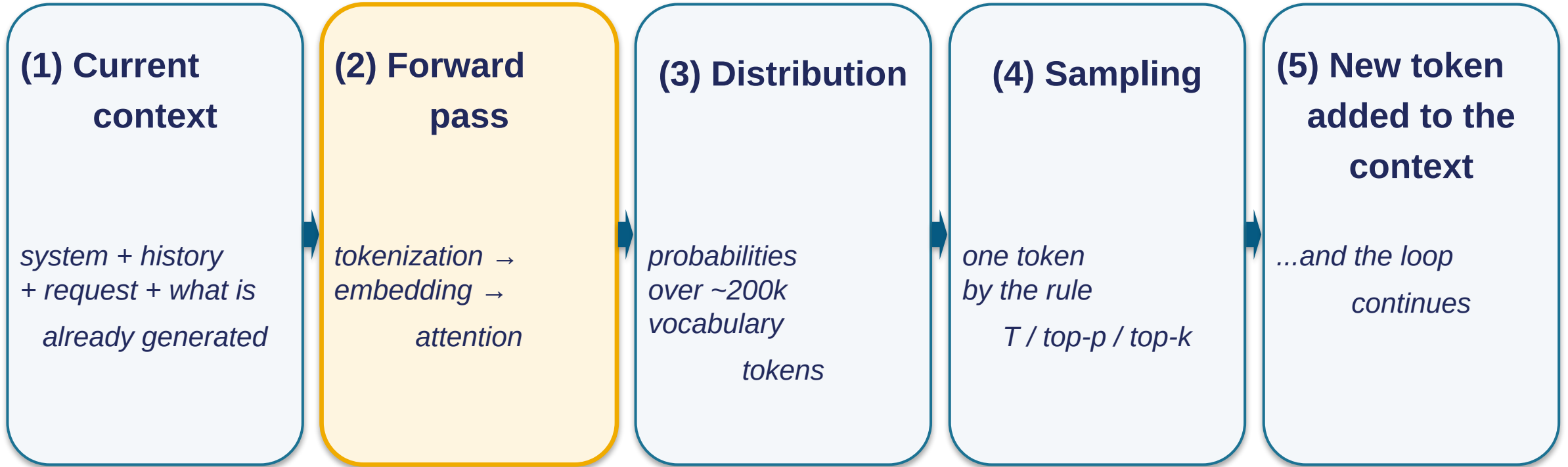
Pick parameters for the scenario deliberately

Scenario	temperature	top_p	max_tokens	system_prompt
Classification / exact extraction	0	—	50–200	Minimal, with an output schema
Code generation	0.2–0.3	0.9	1000+	Role + repository context
Chat explanation to the user	0.7	0.9	500–1000	Role + audience description
Creative writing	0.9–1.2	0.95	2000+	Role + style description

T = 0 makes the choice nearly deterministic; in production micro-variability from batching is possible — for most tasks it is negligible.

Loop: predicted a token → added it to the context → predict the next

Autoregressive generation — a long answer from stateless calls



↪ **back to step (1) — until we reach the "end of answer" token OR max_tokens**

Each step is stateless. The "memory" of one answer is carried by the context itself, not the model.

Inference is the same locally and in the cloud — but model size determines quality

Architecturally it is the same pipeline. The differences are in size and environment.

Local (Ollama, llama.cpp, LM Studio)

Size: 1–13B parameters

- Qwen 2.5 1.5B • Llama 3.2 1B
- Llama 3.1 8B • Mistral 7B^[1]

● Privacy	<i>requests do not leave for a provider</i>
● Speed	<i>slower on consumer hardware</i>
● Context	<i>a limited window</i>
● Price	<i>0 per token (your own hardware)</i>

Cloud (OpenAI, Anthropic, Yandex, Sber)

Size: 200B+ parameters

- GPT-5, Claude 4.7
- YandexGPT, GigaChat
- Gemini

● Quality	<i>better on hard tasks</i>
● Latency	<i>200–500 ms</i>
● Context	<i>up to 1M tokens</i>
● Price	<i>pay per token, RU ~ 2x EN</i>

[1] [Mistral AI — Mistral 7B](#)

Section 5

Wrap-up

Closing the 3 "whys" + bridge to Lecture 3

0 Intro

1 Tokens

2 Embeddings

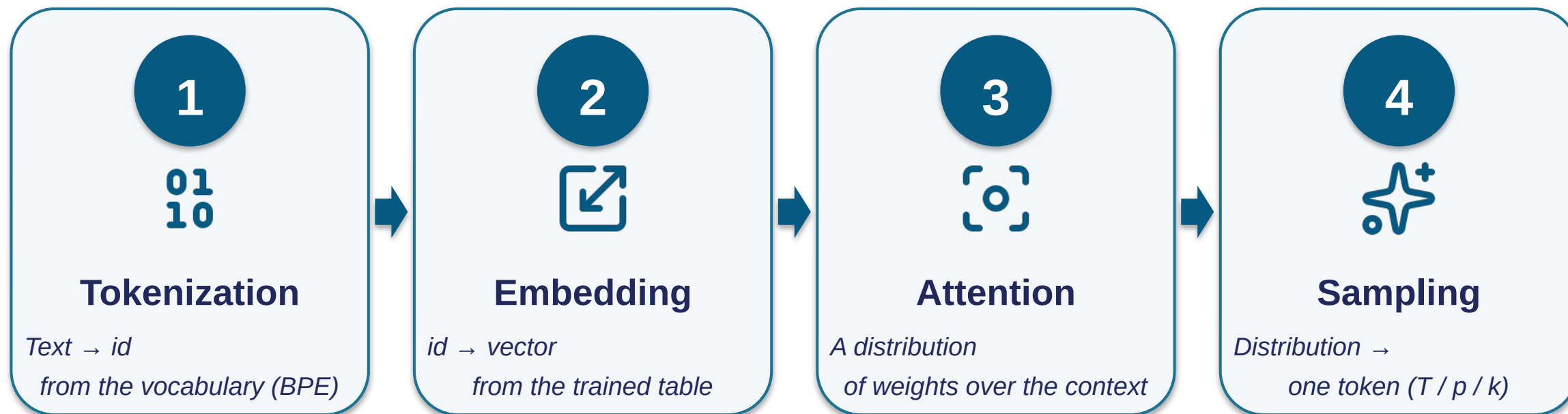
3 Attention

4 Sampling

5 Wrap-up

The 4 inference stages line up into a pipeline

The same black box from Lecture 1 — now unpacked



Lecture 1 called this pipeline the "inference model" — a black box. Now it is no longer black.

Answers to the questions from the start of the lecture

1

Why does a prompt with a role work better than an empty one?

At the attention level, role tokens get high weight — the model leans on them when choosing the next tokens.



2

Why is AI bad at counting letters?

The tokenizer merges letters into tokens. strawberry — 3 tokens, not 10 letters. The model sees tokens, not letters.



3

Why does the same request give different answers?

Sampling — a stochastic choice from the distribution when $T > 0$. Each run may pick a different token.



When an LLM is not the right tool: a decision tree

When is an LLM not the right tool?



Fixed classes

Classification over a small set of categories (5–20)?

→ **Classical ML**
logistic regression, XGBoost,
LightGBM, fine-tuned BERT



Interpretability

*Interpretability needed
(finance, medicine, insurance)?*

→ **Transparent methods**
logistic regression + importances,
decision trees, rules



Response speed

*Response time < 100 ms critical
(anti-fraud, on the users device)?*

→ **A specialized
small model**
(not an LLM ≥ 200 ms)

Otherwise — an LLM fits (chat, RAG, generation, multi-step reasoning)

Attention looks at tokens statistically — it does not understand causality

AI counts correlations in the data, it does not build a causal graph



Human

"X happened because Y"

A causal model — it builds mechanisms.

Relies on physical intuition, domain knowledge, knowledge of how the world works. For causal conclusions this experience is exactly what is needed — not statistics.



AI (via attention)

"X follows Y in the data"

A statistical correlation, not causality.^[1]

Notices the pattern "X and Y often co-occur" in the training data. For causal conclusions — bring in a domain expert or causal methods.

[1] [Pearl \(2018\) — The Book of Why](#)

Lecture 3: "Agents, RAG, API — how AI goes beyond the chat"

All 4 concepts build on top of a single inference pass



RAG^[1]

Retrieval-Augmented Generation

embedding similarity + LLM → an answer from your own base



Tools / Function calling

structured JSON

the LLM generates a call → an external system executes it
→ the result is returned



MCP^[2]

Model Context Protocol

An open standard for connecting tools (Anthropic, 2024)



Agent loop^[3]

act → observe → adjust

the model decides an action, sees the result, adjusts the plan

Q&A

Thank you